Justin Christopher

Enhancement Three: Databases Narrative

The artifact I selected for this enhancement is my Android Inventory App I originally developed in CS 360. The app was built to track inventory levels and demonstrate CRUD functions. It features a login screen where users can login or create an account and an inventory screen where items, quantities, and low-stock thresholds are managed. Rows highlight red when an item falls below its threshold.

Justification

I selected this artifact because the database layer is the foundation that every other part of the application depends on. The users table and items table in DBhelper are what make authentication, inventory management, and the role-based access control system from Enhancement One all possible. Including this artifact lets me show that I can design a schema intentionally and evolve it to meet new requirements without breaking what is already working.

The specific enhancement I made for this category was expanding the SQLite database schema to support the role-based access control system introduced in Enhancement One. In the original application, the users table contained only three columns: id, username, and password. Any registered user was immediately granted full access to all inventory data. My enhancement added three new columns to the users table: a role column that stores either admin or user, an account_status column that stores pending, approved, or denied, and a can_edit column that acts as a boolean permission flag for write access.

To support these new fields, I added several new CRUD methods to DBhelper. The getUserStatus() method checks whether an account is approved before login is allowed. The

approveUser() and denyUser() methods give the admin the ability to control access. The setPermission() method toggles the can_edit flag, and getRole() retrieves the role of a logged-in user so InventoryActivity can show or hide write controls appropriately. I incremented DB_VERSION from 1 to 2 and updated the onUpgrade() method to handle migration. A default admin account is seeded automatically on the first install so the workflow is testable out of the box. Enhancement Two also added a searchItems() method to DBhelper that builds a dynamic SQL WHERE clause using LIKE for name filtering and a qty condition for the low-stock filter, which pushed filtering logic to the database engine rather than loading all rows into memory and filtering in Java.

Course Outcomes

I planned on meeting course outcome three with this enhancement, which asks students to design and evaluate computing solutions that solve a given problem using algorithmic principles and computer science practices and standards appropriate to the solution, while managing the trade-offs involved in design choices. I believe I met this outcome. The schema expansion required evaluating trade-offs between simplicity and correctness. Adding role and account_status as separate columns rather than collapsing them into a single field makes the data more query-able and the logic easier to reason about. Seeding a default admin account at database creation rather than requiring a manual insert is a design decision that prioritizes usability without weakening the security model. The searchItems() method reflects the same kind of thinking, since pushing filtering into SQL rather than doing it in Java is the more algorithmically sound choice even though it requires more careful query construction.

I do not have any updates to my coverage plan. The enhancement went as described in module one.

Reflection

The most important thing I learned working on this enhancement was how interconnected the database layer is with the rest of the application. Adding three columns to the users table had ripple effects across LoginActivity, InventoryActivity, AdminActivity, and multiple DBhelper methods. That experience reinforced why schema decisions matter upfront, because changing them later is not just a database problem but a whole-application problem.

One challenge I ran into was the same silent schema failure I described in Enhancement One, where a missing space in the CREATE TABLE statement caused the table to be malformed without throwing a clear error. The admin seeding failed silently and it looked like the login logic was broken until I traced it back to the schema. Another challenge was the validateUser() method. The original version returned a simple boolean, which was fine when any valid login meant full access. With the new account_status field, the login screen needed to distinguish between a wrong password, a pending account, and a denied account and show a different message for each. Updating the method to pass back enough information for the UI to respond correctly required rethinking the return type and updating every call site, but the result is a cleaner interface between the database layer and the activity layer.